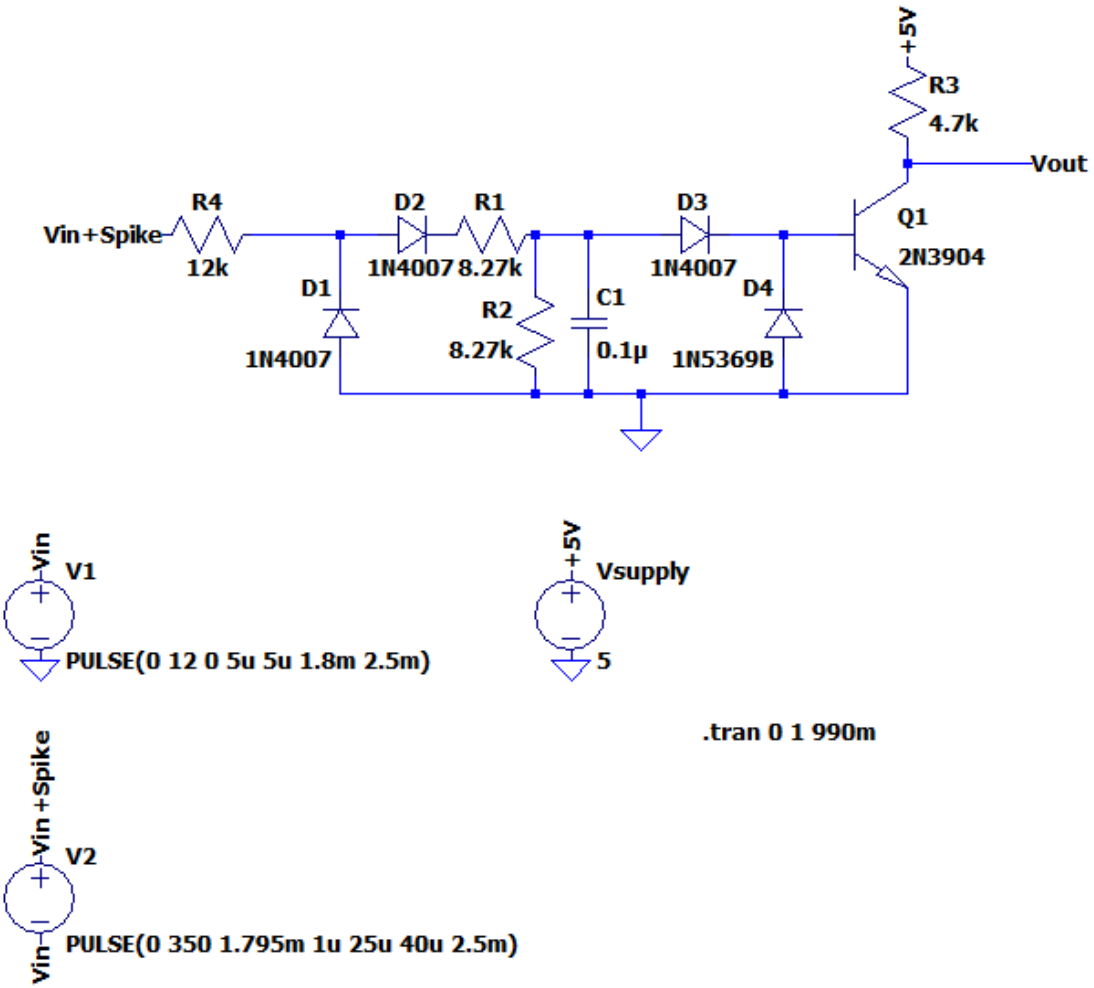
Tachometer Circuit

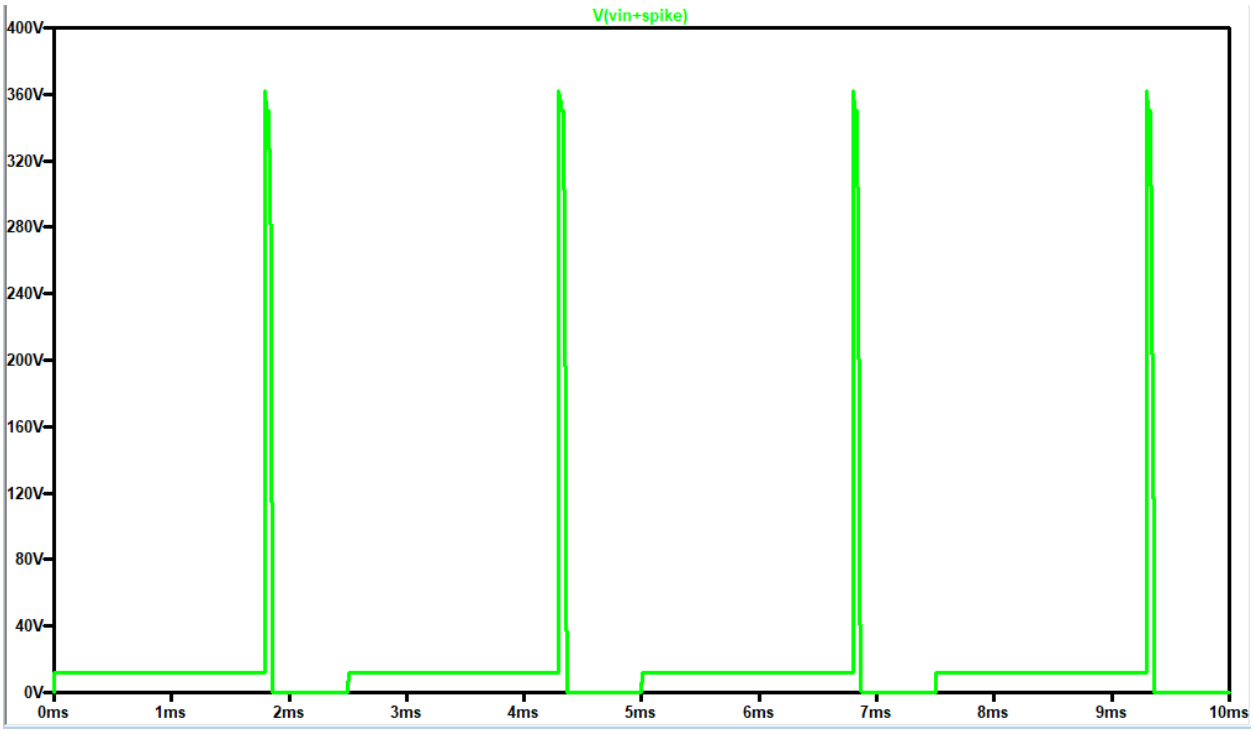
Jonathan Meeker

1. Purpose
    a. I need a tachometer for my 1967 mustang as it doesn't come with one. The price for buying one off the market is ridiculously high, so I figured I'd just make it
2. Design
    a. Overview: I can electronically read the rpm of my car by reading when the negative terminal of the coil activates each spark plug. I then want to output this reading using a needle gauge that I can mount on my dash
    b. Circuit design: The main issue with reading whenever the coil fires is that it needs to generate a high voltage in order to make the spark plugs fire. The coil stays around 12V in idle, and then drops to 0V when it fires. On the falling edge of that square wave, the spark ignites increasing the voltage to around 300-400V. This could damage any signal reading device I use to measure the frequency, so to avoid this I must design a circuit to handle the high voltage

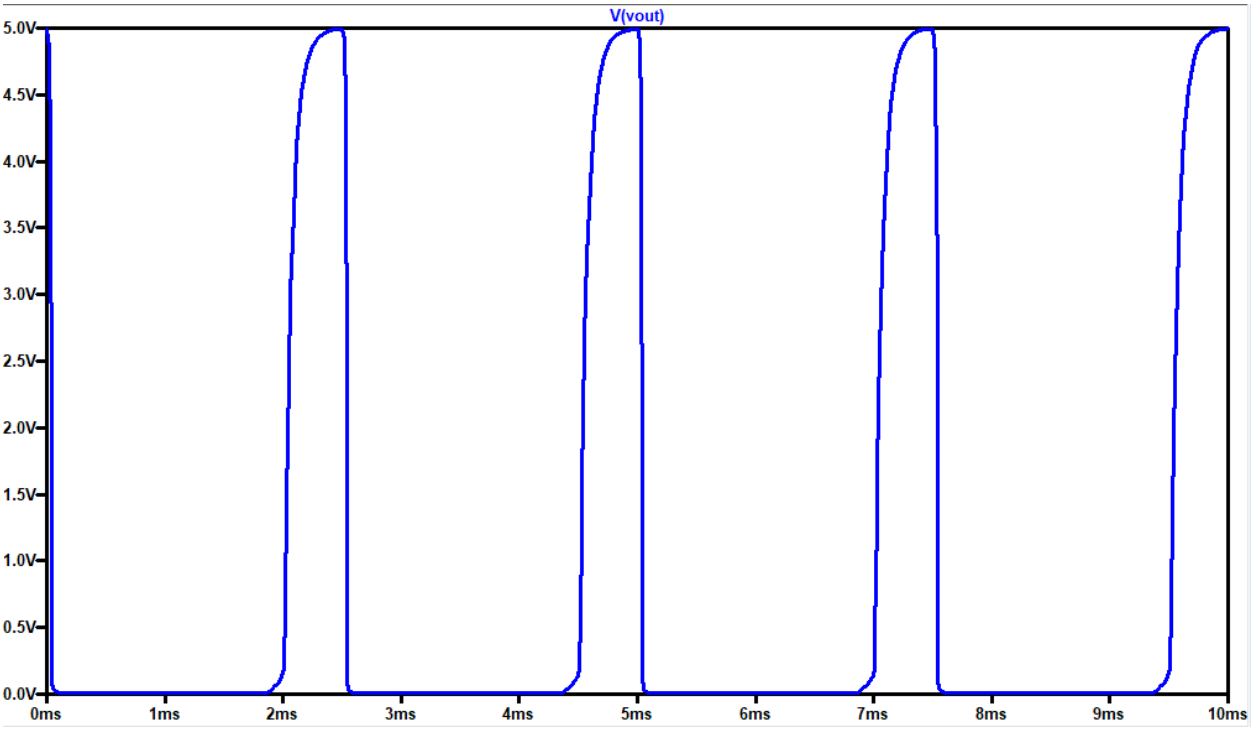


This circuit clamps the input voltage to around 6.1 Volts, which is pretty safe for an Arduino to read, while also filtering out EMI noise from the engine bay and handling the large voltage spike. The R4 input resistor helps reduce the current input to the circuit. The D1 diode prevents negative voltage from entering the transistor base or the Arduino. R1 and R 2 voltage divide to further reduce incoming current and voltage. And the clamper circuit ensures no over voltage can damage any of the components. Finally there is a pull up resistor to 5V to ensure no ambiguity for the Arduino to read when it fires. I'm simulating the coil spark and frequency with a 12V to 0 V square wave and a 350V spike on the falling edge of the circuit (which is when the spark plug fires)

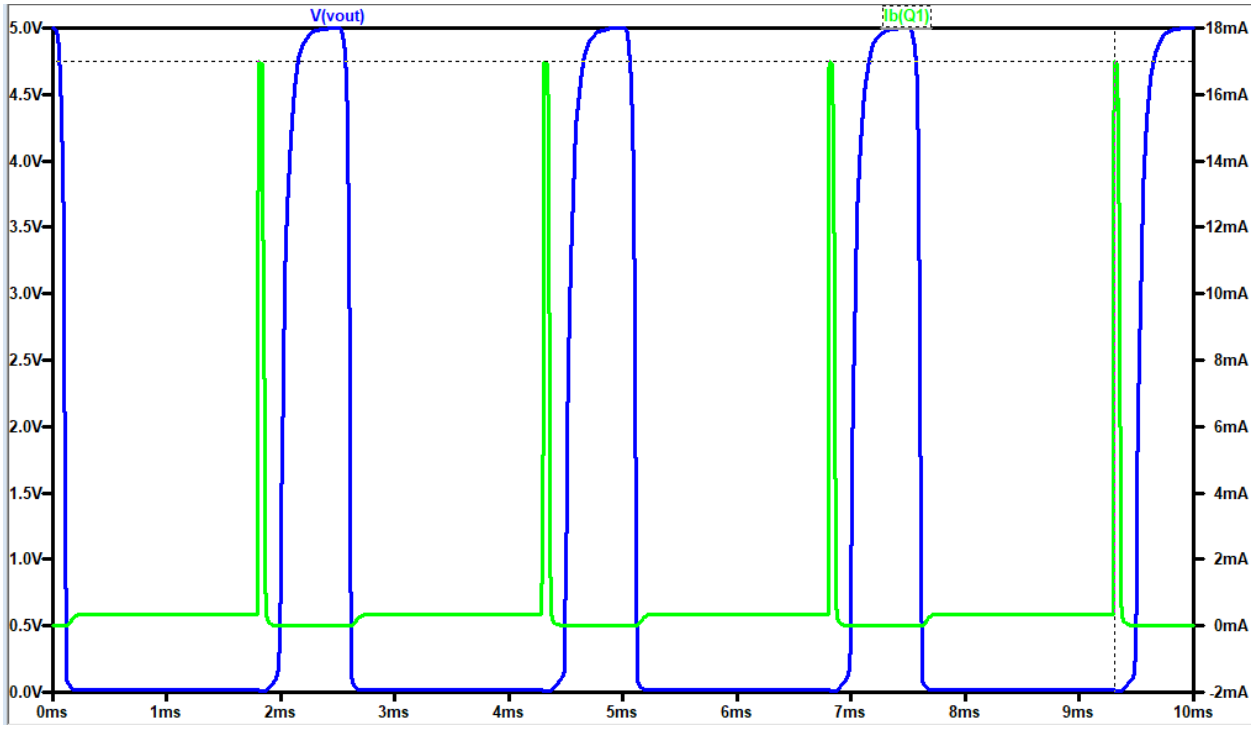
Simulated coil input for the circuit:



Simulated output of the circuit to the Arduino:

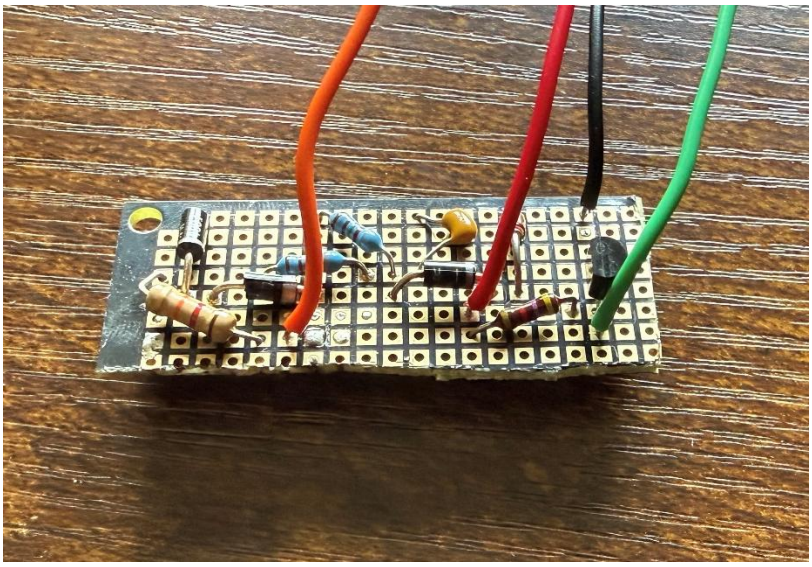


Simulated output and max current into the base of the transistor:



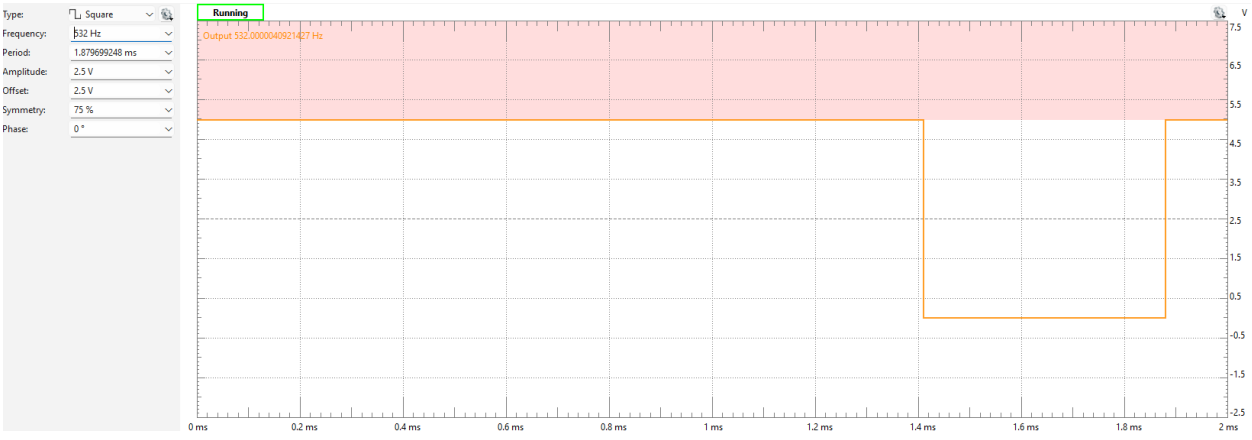
As expected, the circuit outputs a safe 5V to the Arduino and handles current appropriately (with a max current output of 17mA, well under the max peak current of around 100mA for the 2n3904)

3. Progress
    a. Built circuit

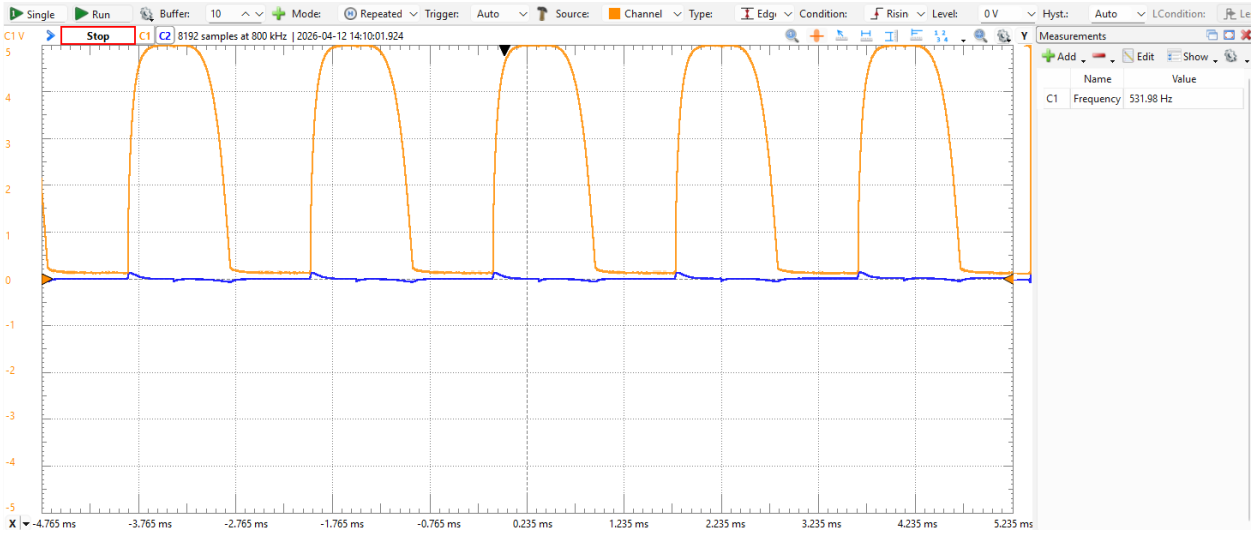


This circuit successfully steps down a high input voltage and filters out noise. There are no concerns with the high spark voltage affecting Arduino or NPN transistor.

Input wave shape:



Output for a 532Hz 5V to 0V square wave:



The output looks relatively clean and will be able to interpret the frequency very easily on the Arduino. Note that 532Hz is equivalent to a max 8k RPM, which is the highest that I plan to output on this device

Now for the mistakes. I made a couple errors on this part, but the biggest one is I didn't test the circuit on a breadboard first.

i. Mistake #1: Not testing
    1. Error: jumping to soldering. Fix: test first
        a. I kept iterating over and over again because I realized the built circuit was not exactly like simulated circuit. I had to redo the current calculations to get it to a safe current, and then I had issues with the RC delay attenuating the transistor output, and then I realized that all of it was fine because frequency is measured in seconds, not minutes, so the high frequency that was causing the signal to attenuate is no where near the max the system would have to output. In summary, next time test on breadboard first, make all your mistakes and fix them, and then solder.

ii. Mistake #2: Messy soldering
    1. Error: sloppy work. Fix: Do better
        a. Not a lot to say here. I had to keep reiterating on my design which required me to desolder and resolder a bunch of times and left a lot of mess on the board. In

the future I account for all of the variables before soldering

4. To do:
    a. Code gauge unit
    b. Assemble Arduino, Voltage limiter circuit, and gauge dial
    c. Test and Verify